Inventory Reconciliation Agent

Application Guide

How the application works: the business problem, the process, the three-layer engine and the screens

Item	Value
Plant	Bimbo Bakeries USA, Lehigh Valley bakery (Breinigsville, PA)
Application	Inventory Reconciliation Agent: exception worklist, source simulator, reports
Version and phase	0.2.0 · Phase 1 proof of concept
Readers	Business and technical readers. Shaded **Technical detail** boxes can be skipped by business readers.
Date	04 November 2026

All screen images and examples in this guide use the simulation's invented data (item numbers 9001xx, names starting with “SIM”). The validation figures in section 1 come from one day of real plant data.

Contents

1. Summary . . . 4

The three layers . . . 4

Where the project stands . . . 4

Result on one day of real data (Saturday 12 September 2026) . . . 4

2. The business problem . . . 5

2.1 The four systems . . . 5

2.2 Why the systems disagree . . . 5

2.3 What it costs . . . 5

2.4 The eight known problems . . . 6

3. How the application works, end to end . . . 7

3.1 The reconciled position . . . 7

3.2 How records in different systems are matched . . . 8

4. The data . . . 9

4.1 Two ways data arrives . . . 9

4.2 The five daily extracts . . . 9

4.3 Data quality the agent handles . . . 9

5. Architecture: the three layers . . . 10

5.1 Layer 1: the rule-based engine . . . 10

5.2 Layer 2: the statistical and learning layer (ML layer) . . . 12

5.3 Layer 3: the descriptive AI layer . . . 14

6. Using the worklist . . . 16

6.1 Sign in . . . 16

6.2 The main screen . . . 17

6.3 Reading a row . . . 18

6.4 What people do with a flag . . . 18

6.5 Staying current . . . 19

6.6 Corrections (write-back) . . . 19

7. The source simulator . . . 21

7.1 Tables and the row editor . . . 21

7.2 The change list . . . 22

7.3 Making the integration misbehave . . . 22

7.4 Scenarios . . . 23

7.5 How one change travels . . . 23

8. The practice day and the eight scenarios . . . 24

9. Roles and responsibilities . . . 25

Outputs and reports . . . 25

10. Technology and operations . . . 26

Security . . . 26

Running it . . . 26

11. Open questions for Bimbo IT . . . 27

12. Limits today and the Phase 2 plan . . . 28

Limits of Phase 1 . . . 28

Phase 2 plan . . . 28

13. Glossary . . . 29

Appendix A. Rule conditions and calculations . . . 30

Quantities per product . . . 30

Rule conditions . . . 30

Related documents . . . 31

1. Summary

Bimbo's Lehigh Valley bakery runs on four systems that describe the same bread: **MIPS** plans what to make, **ODI / Oracle** records what was made, **WMS** stores and ships it, and **Oracle Fusion** keeps the books. They often disagree, and today supervisors find the disagreements by comparing screens by hand.

The **Inventory Reconciliation Agent** does that comparison automatically, for every product, every time the data changes. Each disagreement becomes a **flag** that says what is wrong, in which system, how many units and dollars are at stake, and who should fix it. People work through the flags on one web page, the **exception worklist**.

The three layers

- **Layer 1: Rule-based engine.** 19 fixed checks compare the systems and calculate every number. This is where the business knowledge lives: each rule encodes one known way the systems drift apart.
- **Layer 2: Statistical and learning layer (ML layer).** Robust z-scores (median and MAD) find values that are far outside the plant's normal range, such as a keying error. The layer also learns from planner decisions which rules to trust, and ranks the flags.
- **Layer 3: Descriptive AI layer.** A large language model (Gemini 2.5 Pro on Oracle Cloud) turns the flags into plain explanations, a root cause, an owner and a shift briefing, and answers questions.

The rules calculate; the AI only explains. Every quantity comes from layers 1 and 2, with the source values (the evidence) attached. The AI never calculates or changes a number, and the application works fully without it.

Where the project stands

- **Phase 1 (now):** reads the plant's five daily Excel extracts and recommends corrections, which planners make in the source systems.
- **Simulation:** a **source simulator** plays the four systems with invented data, to show real-time behaviour, integration failures and write-back (sending an approved fix back to the source system).
- **Phase 2 (planned):** live connections to the plant systems, SMS and Teams alerts, real write-back.

Result on one day of real data (Saturday 12 September 2026)

Measurement	Result
Lines / runs / batches / WMS orders read · products compared (with no mismatch)	157 / 86 / 36 / 18 · 78 (31)
Flags found	104 (8 High, 76 Medium, 20 Low)
Retail value at risk	about $227,000
Units at risk of over-production · surplus not carried forward	12,952 · 35,205
Units shipped from stock MIPS cannot see · orders shipped with no load scan	2,571 · 18 of 18

These are findings from one sales day, not measured savings. Savings can only be measured after the plant uses the agent for a longer period.

2. The business problem

2.1 The four systems

System	Role at the bakery	What the agent reads from it
MIPS	In-house production planning. Holds what must be made for each sales day: customer sales orders (sent in by **BUSS**) and supply-plan lines (sent in by **OCloud**, Oracle supply planning). It also nets leftover inventory against demand before sending orders to production.	Schedule lines: item, quantity, sales day, source, units per tray, price
ODI / Oracle production	Records production. For each run: how much it was ordered to make and how much was actually wrapped (packed). Also the dough batches mixed.	Finished-good runs and batches
WMS	The warehouse. Customer orders, what is set aside (allocated) for each customer, receipts from the plant, shipments, returns and freezer stock.	Orders by item and customer, order headers and shipping status; in the simulation also receipt tickets and stock movements
Oracle Fusion	The company's inventory and finance system of record. Every warehouse movement must be posted here.	Postings of warehouse movements (simulation only today)

2.2 Why the systems disagree

Each system is updated by different people and different interfaces, at different times. The gaps open at the handoffs:

- **Timing:** a customer changes an order in WMS after MIPS last refreshed.
- **Planning logic:** MIPS nets leftover stock against only the first order line of a product.
- **Physical events nobody records:** bread taken from the freezer, damaged bread sent back, a run moved to another line.
- **Interfaces that fail:** a warehouse movement never reaches Oracle, or Oracle rejects it.
- **Master data:** MIPS and WMS disagree on how many units fit in a tray.
- **Formatting:** one system pads item codes with spaces, so exact matches fail.

2.3 What it costs

- **Over-production:** the plant bakes bread it already has.
- **Wasted surplus:** extra bread stays on one sales day instead of reducing the next day's baking.
- **Wrong stock figures:** freezer stock is used but never deducted, so the records show stock that is gone.
- **Finance cannot close:** shipments missing from Oracle leave the books wrong.
- **Time and knowledge risk:** experienced supervisors spend hours comparing screens, and the knowledge leaves with them.

2.4 The eight known problems

The requirement brief lists eight recurring problems. Each one maps to one or more rules:

#	Known problem	In plain words	Rules
1	Multiple order lines for the same product	Leftover stock is counted against only one order line, so the plan asks for too much	R01, R06
2	Inventory on a zero-quantity (blind) line	Stock is netted through an empty line and never recorded	R02, R03, R08
3	Inventory does not follow a moved run	A run moves lines; an empty copy stays behind	R04
4	Additional (recovery) production runs	A top-up run re-makes the whole order, not just the gap	R05, R14
5	Net quantity not reflected	Returned or damaged bread is still counted as received	R10, R19
6	Screen refresh / stale data	An order changes in one system and not the other	R09
7	Freezer inventory remains after use	Freezer stock is used but not deducted	R07
8	Missing downstream transactions	Shipments or postings do not reach the next system	R12, R13, R16, R18
–	Data quality	Different units per tray, padded codes, unusual values	R11, R15, R17

3. How the application works, end to end

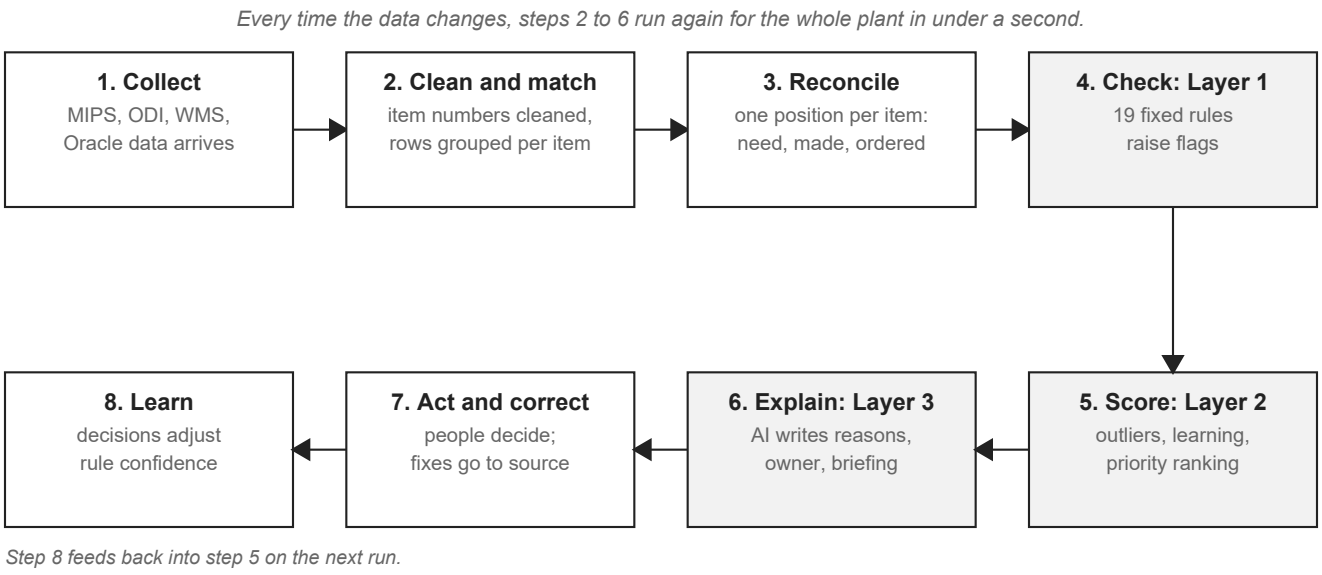


Figure 1. The process. Shaded boxes are the three layers.

1. **Collect.** Data arrives from the four systems. In Phase 1 these are five daily Excel extracts; in the simulation (and in Phase 2) it is a stream of row-level changes.
2. **Clean and match.** Item numbers are cleaned (spaces, leading zeros and Excel's “.0” removed), so the same product matches in every system. Rows are grouped per product.
3. **Reconcile.** For each product the agent builds one **reconciled position**: how much is needed, how much stock was applied, how much was made, how much customers ordered and how much was set aside.
4. **Check (Layer 1).** 19 rules compare the systems and raise a flag for every disagreement.
5. **Score (Layer 2).** Statistics find unusual values; learned rule confidence and dollar value rank the flags.
6. **Explain (Layer 3).** The AI writes a plain explanation, root cause and owner for each flag, and a shift briefing.
7. **Act and correct.** On the worklist, people claim, acknowledge, accept or reject flags. Corrections are made in the source system (by hand in Phase 1; by approved write-back in the simulation).
8. **Learn.** Every accept or reject updates the rule's confidence, which changes the ranking on the next run.

3.1 The reconciled position

The heart of the application is one equation per product and sales day, the same one a planner would use:

```
Net requirement  =  Demand (all MIPS lines)
                 −  Inventory applied (stock MIPS netted out)
                 −  Completed production (ODI wrapped, all runs)
                 +  Returns to plant (WMS)

Positive: still to produce (or supplied from stock MIPS cannot see)
Negative: surplus to carry forward to the next sales day
```

Why this matters: MIPS applies leftover stock line by line. The agent adds **all** lines of a product first and then applies the stock. This one change removes the most common cause of over-production (rule R01).

Worked example: whole-wheat bread (item 900101)

Step	Value
Demand: three MIPS lines, 1,300 (moved from the day before) + 2,400 + 7,600	11,300
Sent to production by MIPS (one run)	9,000
So stock that MIPS applied = 11,300 − 9,000	2,300
What the plant should make = 11,300 − 2,300	9,000
What MIPS asks for if the 2,300 is applied to the first line only (1,300): 11,300 − 1,300	10,000
Risk of over-production = 10,000 − 9,000 → flag R01, High, 1,000 × $4.69	1,000 units · $4,690

3.2 How records in different systems are matched

The systems share no common line number. MIPS does not know which customer an order line is for, and a production run does not say which order line it was made for. So the agent matches at the level that all systems share: the **product (item number)**. It adds up each system's rows per product and compares the totals. A few links also connect individual rows:

Link	Matched on	Used by
Production run ↔ dough batch ↔ receipt ticket	the run (formula) code	R04, R14
WMS order header ↔ its item lines	the customer name	R13
Receipt ticket ↔ return to plant	the ticket number	R19
Warehouse movement ↔ Oracle posting	the movement number, stored in Oracle as its reference	R18

What this means: the agent can say “potato bread was over-built by 1,200”, but not “order line X was made by run Y”, because the source data does not record that link. Where line-level detail is useful, the agent *proposes* an allocation by rule: stock first, then production, earliest sales day first, customer orders before forecast lines.

4. The data

4.1 Two ways data arrives

	Daily Excel extracts (Phase 1, real data)	Change feed (simulation now, Phase 2 later)
Source	Five files the plant exports per sales day	Every row-level change in any system, as it happens
Timing	Once a day; re-read automatically when a new file arrives	Within a second of the change
Content	Order lines, production runs, batches, order headers, orders by item	The same five tables, plus receipt tickets, warehouse movements and Oracle postings
Rules available	R01–R17	R01–R19 (R18 and R19 need the extra real-time detail)
Corrections	Recommended; planners make them by hand	Can be approved and sent back (write-back)

Technical detail

Both paths produce the same internal data set, so the engine does not know or care where the data came from. The simulated feed uses the client's own column names for every table, and the same code reads both. The Excel reader finds each file's header row, drops the report title rows and totals rows, and flattens the block layout of the WMS item report. The feed is an append-only event log (SQLite in the proof of concept, OCI Streaming planned) with full before and after images of each row; duplicates and out-of-order deliveries are detected and ignored.

4.2 The five daily extracts

File	System	One row is
MIPS Order Data	MIPS	one planning line: a customer order or a supply-plan line for one product
ODI Finished Good Data	ODI / Oracle	one product made in one production run
ODI Batch Data	ODI / Oracle	the dough mixing for one run
WMS Fill Rate Report	WMS	one customer order (header) with its shipping progress
WMS Order Summary Report by Item	WMS	one customer's order for one product

4.3 Data quality the agent handles

- Item codes padded with spaces (MIPS sales-order lines), stored as numbers in one system and text in another, or with leading zeros. All are cleaned before matching, and rule R15 reports the padding.
- Orders moved from one sales day to another keep their original schedule date; the agent uses the *offset* sales date so moved lines are not double-counted.
- Report layouts with title rows, subtotals and a totals row; the totals are kept and checked (R13).

5. Architecture: the three layers

The engine is split into three layers so that each does what it is best at. Numbers must be exact and auditable, so a fixed rule engine produces them. Some problems cannot be written as a fixed rule, so a statistical layer looks for anything abnormal and learns from people's decisions. People need to understand and act quickly, so an AI layer turns the results into plain language.

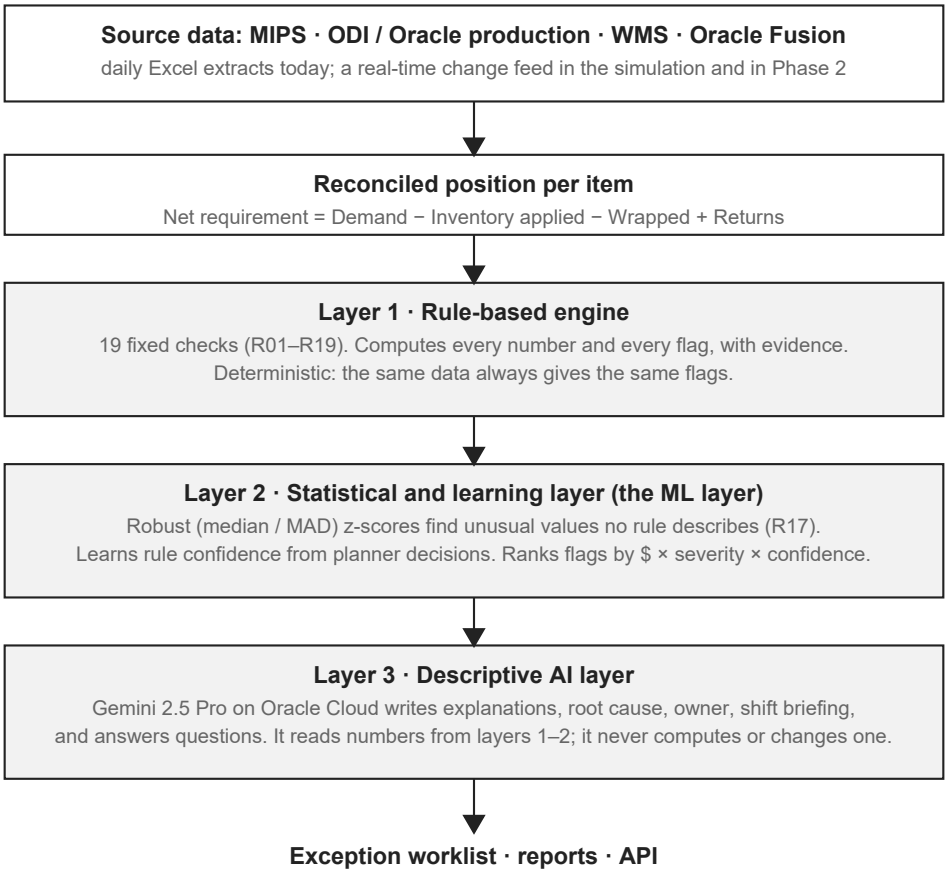


Figure 2. The three layers, from source data to the worklist.

5.1 Layer 1: the rule-based engine

Layer 1 is a list of 19 fixed checks, like the checklist an experienced planner would run, applied to every product every time the data changes. Technically it is simple: each rule is a short, readable function that compares numbers. The complexity is in the **business knowledge** the rules encode: which handoff fails, how it shows up in the numbers, and what the correction is.

Each flag it raises carries:

- the **rule** and the **product** (or the whole plant);
- the **units** involved and the **dollar value** (units × retail price);
- a **severity**: High, Medium or Low;
- the **evidence**: the actual values from each system that triggered it;
- a **recommended correction** written by the rule, and the **systems** involved.

The 19 rules

Rule	What it finds	Severity	Usual owner
R01	Leftover stock counted against only the first order line, so MIPS asks for more than needed	High at 1,000+ units, else Medium	Production planner
R02	MIPS netted stock out of demand, but nothing records where that stock is	Medium	Production planner
R03	A “blind” line: zero ordered and zero made inside a live run	Low	Production planner
R04	A run moved to another line and left an empty “ghost” copy	Medium	Production planner
R05	A recovery (top-up) run made far more than the shortfall	High at 1,000+, else Medium	Shift supervisor
R06	Surplus not carried forward to the next sales day	Medium at 1,000+, else Low	Production planner
R07	The warehouse filled orders with stock MIPS cannot see (often the freezer)	High at 500+, else Medium	Warehouse / WMS
R08	Stock already exceeds demand; the run shows a negative order	Medium	Production planner
R09	MIPS and WMS disagree on how much customers ordered	Medium	IT / integration
R10	The warehouse set aside less than was ordered	Medium	Warehouse / WMS
R11	MIPS and WMS disagree on units per tray	Medium	Master data
R12	Orders shipped with no trailer load scan	High	Warehouse / WMS
R13	Two WMS views of the same orders do not add up	Medium	IT / integration
R14	More or fewer dough batches mixed than scheduled	Low	Shift supervisor
R15	Item codes formatted differently between systems	Low	IT / integration
R16	ODI's copy of the MIPS order quantity is out of date	Medium	IT / integration
R17	A value far outside the plant's normal range (from Layer 2)	Medium; High at z 7+	Shift supervisor
R18	A warehouse movement not posted in Oracle within 10 minutes (live feed only)	High	IT / integration
R19	A return to the plant not subtracted from the receiving ticket (live feed only)	High at 500+, else Medium	Warehouse / WMS

Worked example: a recovery run that over-builds (R05, potato bread 900102)

Step	Value
Needed (two MIPS lines: 1,000 + 5,000)	6,000
Run 1: ordered 6,000, wrapped 5,200, so still needed	800
Run 2 (recovery): ordered and wrapped	2,000
Total wrapped = 5,200 + 2,000	7,200
Over-build = 7,200 − 6,000 (rule fires at 100 or more)	1,200
Severity: 1,000 or more → High. Dollar value 1,200 × $4.69	$5,628

The flag's recommendation reads: *“After run 1 wrapped 5,200, only 800 units were still needed, but the recovery run(s) ordered 2,000 … Apply the 1,200-unit surplus to the next sales day for this SKU (or designate to freezer) and size future recovery runs to the remaining shortfall only.”*

Technical detail

Rules are plain Python functions in `recon_agent/detectors.py`; each takes the reconciled position of one product (or the whole data set) and returns zero or more findings. Quantities must match exactly (tolerance 0 units). Fixed thresholds: R05 needs an over-build of 100+ units, R06 a surplus of 50+, R07 a gap of 20+; R18's posting window is 10 plant minutes. Each flag has a stable ID (rule + key, e.g. `R05-900102`), so decisions survive reloads. The exact trigger condition of every rule is in Appendix A.

5.2 Layer 2: the statistical and learning layer (ML layer)

Layer 2 does three jobs: it finds **unusual values** that no fixed rule describes, it **learns** from planner decisions, and it **ranks** the flags. It uses well-established statistics rather than a trained black-box model. That is deliberate: it works from a single day of data, needs no training set, and every score can be checked by hand.

Job 1: finding unusual values (rule R17)

Most production runs wrap almost exactly what they were ordered to make. A run that wraps three times its order is probably a keying error, and if nobody catches it, the "surplus" is carried forward as stock that does not exist. Layer 2 measures how far each value is from the plant's normal, and flags the ones that are far outside it. It looks at two measures:

- **Wrap overrun per run** = (wrapped − ordered) ÷ ordered. 0% means the run made exactly what was asked.
- **Final piece variance per batch**: pieces over or under the theoretical count.

The ordinary z-score. The usual way to say "how unusual" is the z-score: the distance from the average, measured in standard deviations.

```
z  =  (value − mean) ÷ standard deviation
```

Its weakness is that one bad value pulls the average towards itself and inflates the standard deviation, so it **hides itself** (statisticians call this masking). With small, skewed plant data that is exactly the situation we are in.

The robust (MAD) z-score. Layer 2 uses the modified z-score of Iglewicz and Hoaglin instead. It replaces the mean with the **median** (the middle value) and the standard deviation with the **median absolute deviation (MAD)**, neither of which a single bad value can move much.

```
MAD    =  median of | value − median |
scale  =  the larger of  1.4826 × MAD   and   1.2533 × mean absolute deviation
robust z  =  (value − median) ÷ scale

Flag (R17) when |robust z| ≥ 3.5  ·  High when |robust z| ≥ 7  ·  needs at least 8 values
```

The constants 1.4826 and 1.2533 put both measures on the same scale as a standard deviation for normally spread data. The second one is a floor: because most runs wrap exactly what was ordered, the MAD is often 0, and dividing by 0 would make every small difference look infinite.

Worked example on the simulation data

The simulation's starting day has 11 runs with an order above zero. Most wrapped exactly what was ordered (0%):

Run	Product	Ordered	Wrapped	Overrun	Robust z	Ordinary z
FG-0002	900102 potato	6,000	5,200	−13.3%	−2.50	−1.48
FG-0008	900106 hot-dog buns	4,000	3,300	−17.5%	−3.28	−2.00
FG-0010	900108 honey wheat	2,500	2,900	+16.0%	+3.00	+2.14
8 other runs				0%	0.00	+0.17

Median 0%, MAD 0, mean absolute deviation 4.26%, so scale = 1.2533 × 4.26% = 5.34%. FG-0008 comes closest (−17.5% ÷ 5.34% = −3.28) but stays under 3.5, so **no R17 flag**. Its shortfall is already explained by rule R07.

Now suppose someone keys run FG-0011 (Italian bread, ordered 1,800) as **5,400** wrapped instead of 1,800:

	Ordinary z-score	Robust (MAD) z-score
Centre	mean = +16.8% (pulled up by the error)	median = 0% (unchanged)
Spread	standard deviation = 58.5% (inflated by the error)	scale = 1.2533 × 22.44% = 28.12%
Score for FG-0011 (+200%)	(200 − 16.8) ÷ 58.5 = **3.13**	200 ÷ 28.12 = **7.11**
Result	under 3.5: **the error is missed**	7 or more: **R17 flag, High**

The R17 flag shows 3,600 units (5,400 − 1,800) and $14,364 at risk, with the evidence “+200.0% against a plant median of +0.0%, robust z = 7.1”. Rule R06 also flags the same 3,600 as surplus, so the supervisor sees both the symptom and the likely cause.

Job 2: learning from planner decisions

Every time a planner accepts or rejects a flag, Layer 2 updates that rule's **confidence**. Rules that planners keep accepting rise in the list; rules they keep rejecting sink. A rule nobody has judged yet scores 1.0.

```
confidence  =  ( (accepted + 1) ÷ (accepted + rejected + 2) )  ÷  0.5
```

Decisions on a rule	Confidence	Effect on its flags
none yet	1.00	ranked on dollars and severity only
3 accepted, 1 rejected	(4 ÷ 6) ÷ 0.5 = 1.33	move up
0 accepted, 4 rejected	(1 ÷ 6) ÷ 0.5 = 0.33	move down
10 accepted, 0 rejected	(11 ÷ 12) ÷ 0.5 = 1.83	move up strongly

A rejected flag whose quantity has not changed is not shown again: the planner has already judged that exact variance. Decisions made in the simulation are stored separately and never affect the confidence learned from real plant data.

> **Technical detail**
> This is a Beta(1,1) prior per rule: the posterior mean acceptance rate, divided by 0.5 so that an unjudged rule scores 1.0 (`recon_agent/feedback.py`). Decisions are stored in a JSON file and applied on every run. The planned Phase 2 gate for automatic write-back (98% acceptance over 8 weeks) would be measured on this same decision record.

Job 3: ranking the flags

```
Priority  =  (dollar value + base value)  ×  severity weight  ×  rule confidence

severity weight:  High 3 · Medium 2 · Low 1
base value:  50 by default · R14 500 · R04 1,000 · R18 2,000 · R12 5,000
```

The base value gives flags that carry no dollar value (a missing load scan, a ghost run) a fair place in the list. Example: the potato-bread R05 flag scores (5,628 + 50) × 3 × 1.0 = 17,034, the highest on the starting day. The worklist's headline **dollar exposure** adds the dollar values of R01, R05–R10 and R19 only, the rules where the units are product genuinely at risk.

5.3 Layer 3: the descriptive AI layer

Layer 3 makes the results readable. It receives the ranked flags from layers 1 and 2, with their evidence, and writes:

- for every flag: a 2–3 sentence **explanation** a supervisor can act on, the most likely **root cause** (which handoff failed and why) and the **owner**, chosen from five roles (production planner, shift supervisor, warehouse / WMS, IT / integration, master data);
- a **shift briefing** of under 150 words and the **3–5 actions** that remove the most risk today;
- **answers** to questions typed into the **Ask** assistant, such as “why is MIPS asking for more 900112?”

What it never does: calculate a quantity, change a number, decide on a flag, or change a source system. It only describes what layers 1 and 2 found.

Guardrails

- The instructions tell the model to use only the numbers it is given or looks up, and never to estimate.
- The answer must follow a strict structure (a JSON schema). An entry with an unknown owner, or an answer that breaks the structure, is thrown away.
- The assistant must look up data with read-only tools before it answers. It cannot write anything.
- Each explanation is labelled **AI explanation** or **Rule text**, so readers always know the source.
- If the model is unavailable, every flag keeps its rule-written text, and the application keeps working.

Timing and cost

The worklist shows rule text immediately, so it is usable at once. The AI text follows. On the live feed it waits for 20 seconds without changes (at most 2 minutes) before calling the model, and it only asks about flags it has not explained before. When the text is ready, the top bar shows **AI explanations ready · Reload**.

The assistant's tools

Tool	What it looks up
search_items	products whose number or description (in any system) contains the text
get_sku_position	one product's full reconciled position: MIPS lines, ODI runs, WMS orders, flags
list_findings	flags, filtered by rule, severity or product
plant_summary	number of flags, units and dollars per rule

Technical detail

Model: Google Gemini 2.5 Pro on OCI Generative AI (on-demand, us-chicago-1), called through the OCI Python SDK (`recon_agent/ai.py`). Narration is one structured call per run for the top 40 flags, with a strict JSON-schema response format. The assistant runs a tool loop of up to 6 rounds; the first round must call a tool. Limits: 32,000 output tokens, 300-second timeout, SDK retry strategy. Explanations are cached and reused for any flag whose ID, units and evidence are unchanged. Setting `RECON_AI=off` disables every model call.

6. Using the worklist

The **exception worklist** is the screen supervisors, planners and managers use. It runs in a web browser.

6.1 Sign in

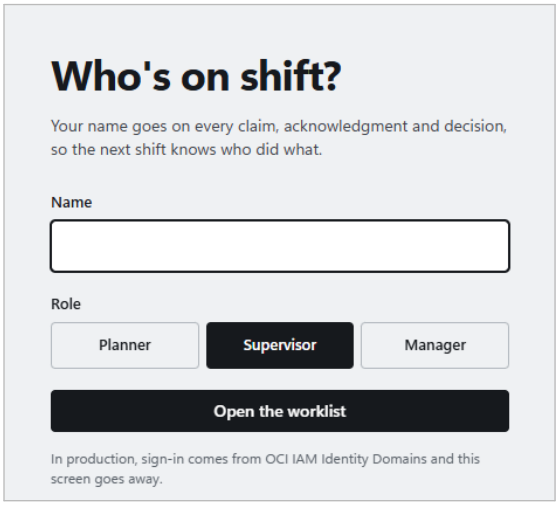


Figure 3. Worklist sign-in.

Each person signs in with a username and password from the administrator. The account sets the role, and the person's name goes on every claim, acknowledgment and decision.

Role	Can do
Planner	Everything, including approving and sending corrections
Supervisor	Read, claim, acknowledge, accept and reject; cannot approve corrections
Manager	Everything, including approving corrections

6.2 The main screen

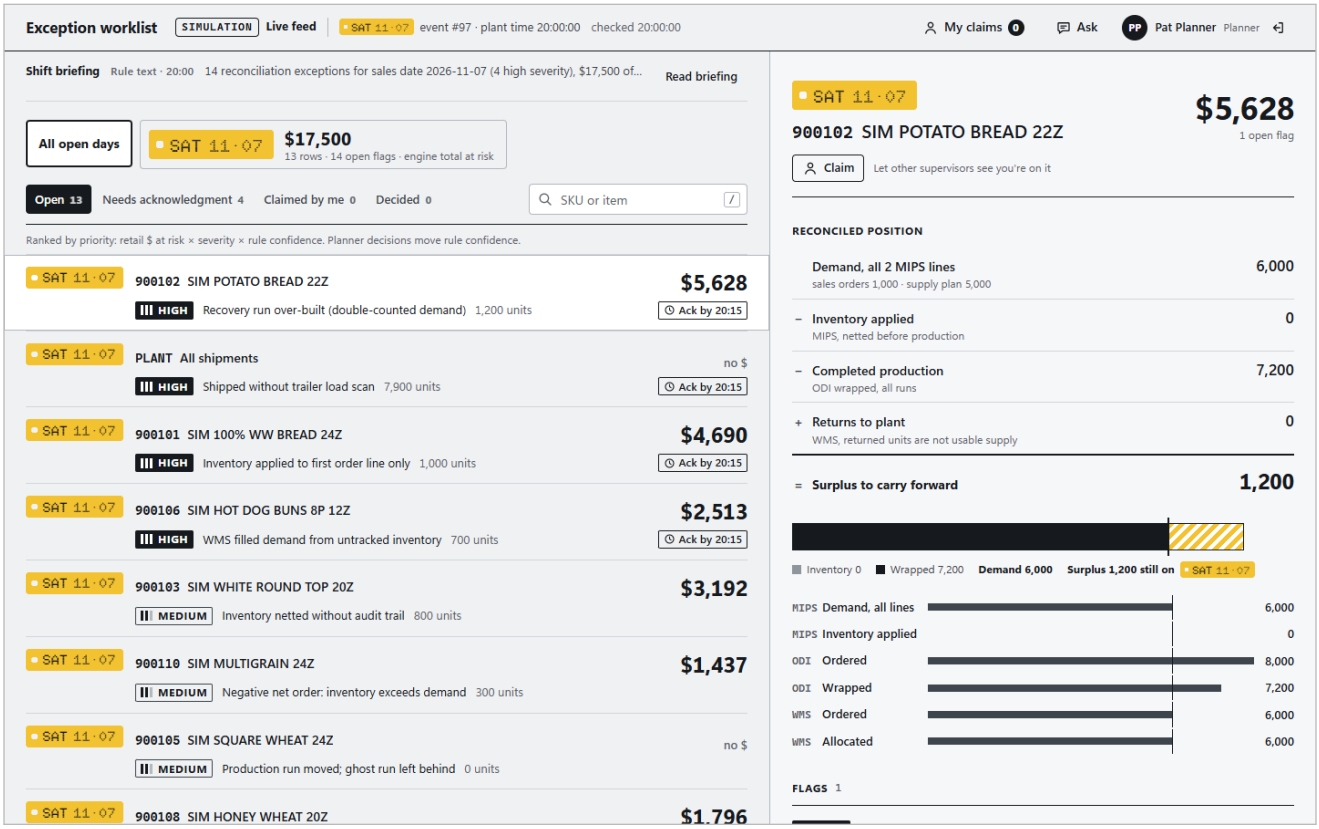


Figure 4. The exception worklist on the simulation's starting day: 13 rows, 14 flags, $17,500 at risk. The potato-bread row is open on the right.

Area	What it shows
Top bar	Data source (**SIMULATION · Live feed**, or daily extracts), the sales day, the last event processed and the plant time; on the right, **My claims**, **Ask** and your name
Shift briefing	A short summary of the situation. **Read briefing** opens the full text and top actions
Sales-day tiles	One tile per open sales day: rows, open flags and dollars at risk
View tabs	**Open**, **Needs acknowledgment**, **Claimed by me**, **Decided**, and a search box
Queue (left)	One row per product with problems (or per plant-level problem), most important first: severity, top problem, units, dollars, and an **Ack by** deadline for High flags
Detail (right)	The selected row: its reconciled position, every flag, the evidence and the action buttons

6.3 Reading a row

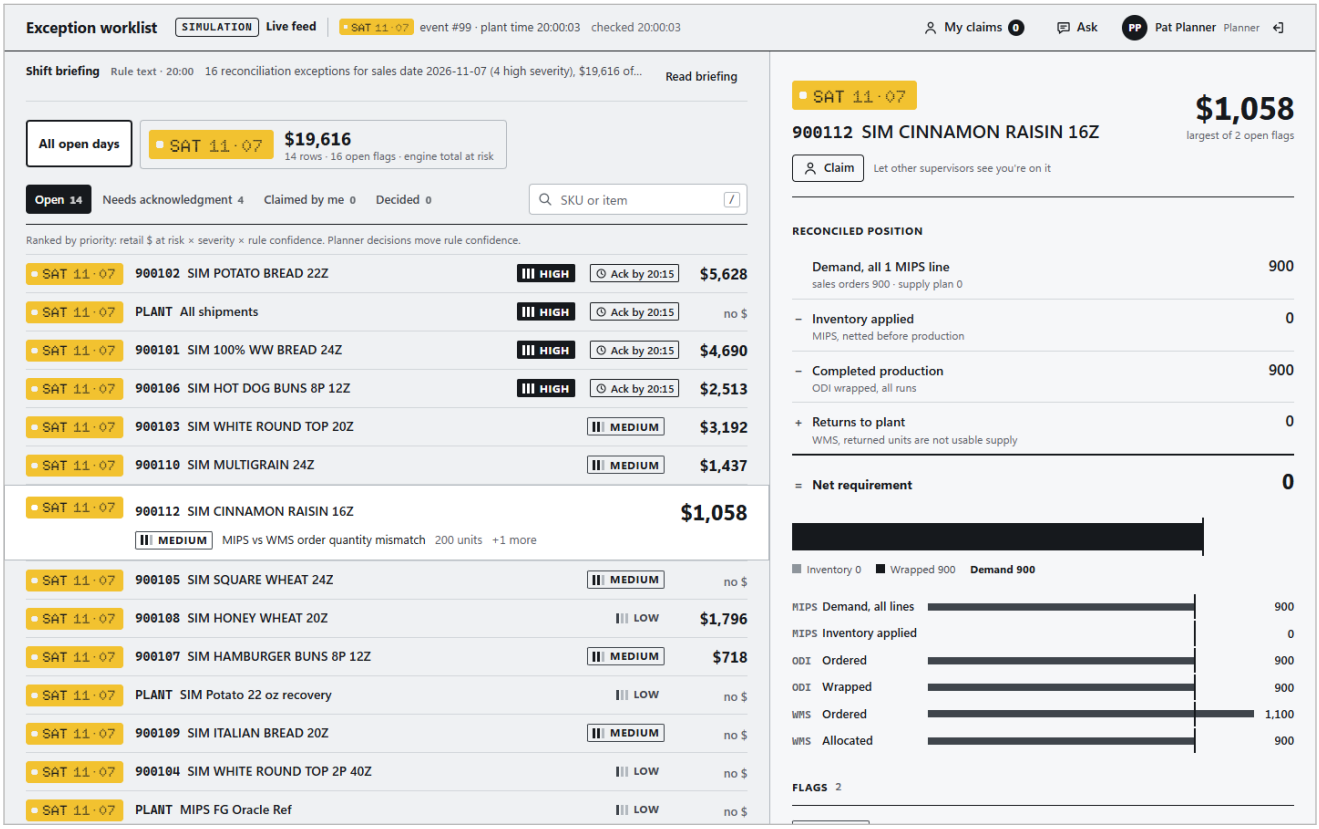


Figure 5. After scenario S6: SIM CINNAMON RAISIN has two flags. WMS has 1,100 on order, MIPS planned 900.

The detail shows the **reconciled position** as a short calculation (demand, minus inventory applied, minus production, plus returns), a bar that draws it, and a table that puts the MIPS, ODI and WMS quantities on one scale, so a mismatch is visible at a glance. In Figure 5, every bar ends at 900 except **WMS Ordered** at 1,100. Below are the flags. Each shows its severity and rule, units and dollars, the explanation (labelled *AI explanation* or *Rule text*), the owner, and the evidence values.

6.4 What people do with a flag

Action	When	Effect
Claim	You are working on a row	Others see your initials on it
Acknowledge	A High flag must be acknowledged within 15 minutes	Records who saw it and when; overdue flags are marked (and escalate to the productivity manager in Phase 2)
Accept	The flag is right	Marked accepted; the correction is due in the source system; rule confidence rises
Reject	The flag is wrong	Rule confidence falls; the flag stays hidden while its quantity is unchanged
Carry forward	Accepting a surplus flag (R05, R06, R08)	Shows the surplus moving to the next sales day, awaiting the MIPS correction
Ask	You have a question	The assistant answers from the current data, with links to rows

Action	When	Effect
Approve and send	A proposed correction exists (planners and managers)	See 6.6

6.5 Staying current

When any data changes, the worklist re-runs every rule and tells every open screen at once. The list does not rearrange itself while someone is reading; instead the top bar shows a button:

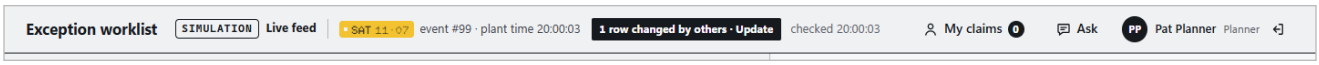


Figure 6. "Rows changed by others · Update". Click it before deciding, so the decision is made on current numbers.

6.6 Corrections (write-back)

For flags where the fix is one clear change to one row in one system, the flag shows a **proposed correction**:

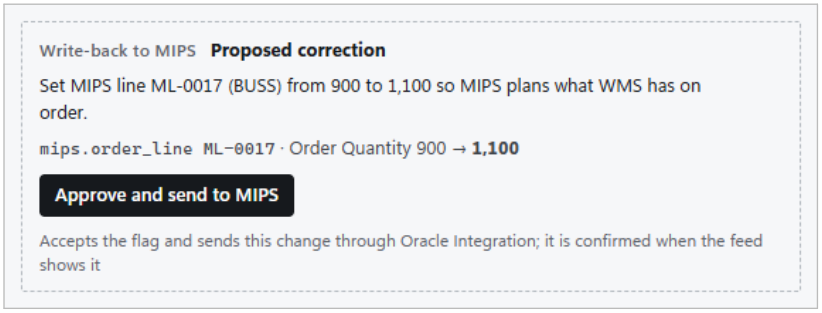


Figure 7. A proposed correction for R09: set the MIPS line from 900 to 1,100 so MIPS plans what WMS has on order.

Rule	Proposed correction
R09	MIPS follows WMS: its main sales-order line takes the difference
R13	Rebuild the WMS order header from its item lines
R16	Re-send the MIPS sales-order quantity to ODI
R18	Resubmit the Oracle posting (or create the missing one)
R19	Book the returned quantity on the receiving ticket

Some flags never get an automatic correction, because the fix needs a fact only a person can confirm: where netted stock physically is (R02), which day takes a surplus (R05, R06), or which freezer lot was used (R07).

Only planners and managers can approve. A correction then moves through these states:

State	Meaning
Approved, sending	Approved; being sent to Oracle Integration
Sent, waiting for the change in the feed	Integration accepted it; the worklist waits to *see* the change arrive
Confirmed in the feed	The corrected values arrived; usually the flag disappears at the same moment
Failed	Integration unavailable or rejected it, or the row changed meanwhile; **Retry** recomputes it
Cancelled	On retry, no correction was needed any more

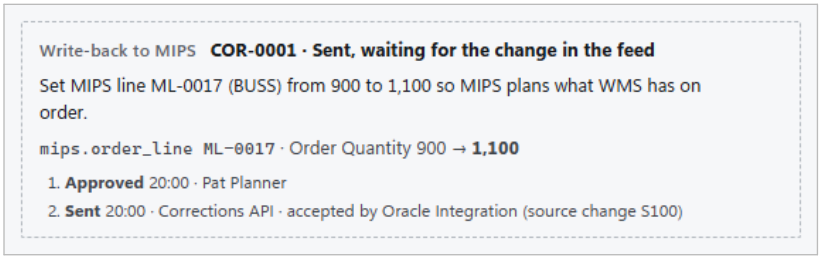


Figure 8. A correction that integration accepted, waiting for the change to arrive. “Sent” is not trusted on its own.

A fix can reveal the next problem. For example, after MIPS raises an order line (R09), ODI's copy of that quantity is now behind, so R16 may appear. That chain is correct, and it is what the simulation is designed to show.

7. The source simulator

There are no live connections to the plant systems yet. To show how the worklist behaves when data changes during a shift, the **source simulator** plays MIPS, ODI, WMS and Oracle with invented data. Think of a flight simulator: the cockpit (the worklist) is real; the world outside the window (the plant's systems) is simulated. The simulator is for demos and testing only and never runs next to a production worklist.

7.1 Tables and the row editor

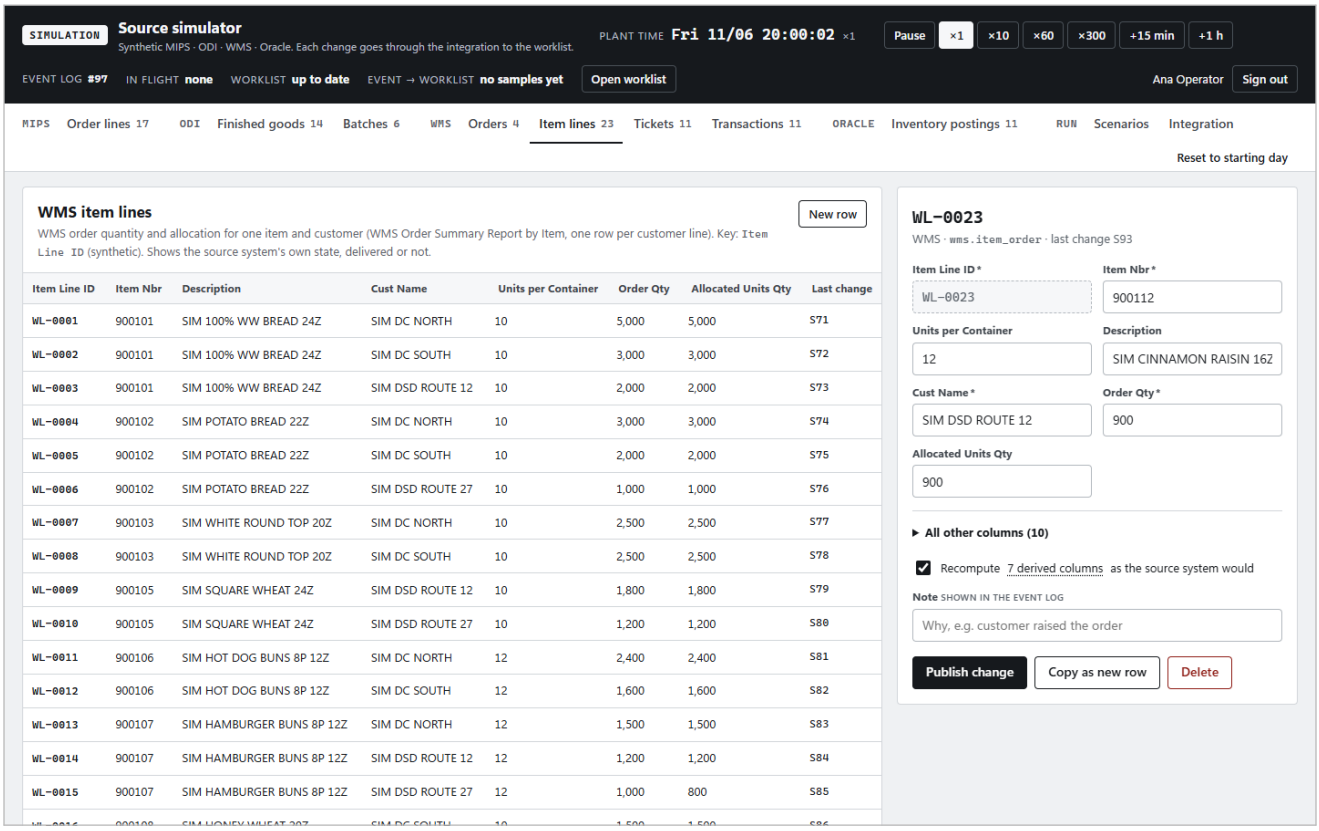


Figure 9. The simulator: one tab per source table (top), the rows, and the editor for row WL-0023 on the right.

Each tab shows one source table as the source system holds it. An operator clicks a row, changes a value and clicks **Publish change**. The simulator recalculates the columns the real system would calculate itself (for example, shortfall and completion status), but it never updates another system: a change in WMS does not change MIPS, just as at the plant. The top bar shows the plant clock (which can run faster), the last event delivered, changes still in flight, and whether the worklist is up to date.

7.2 The change list

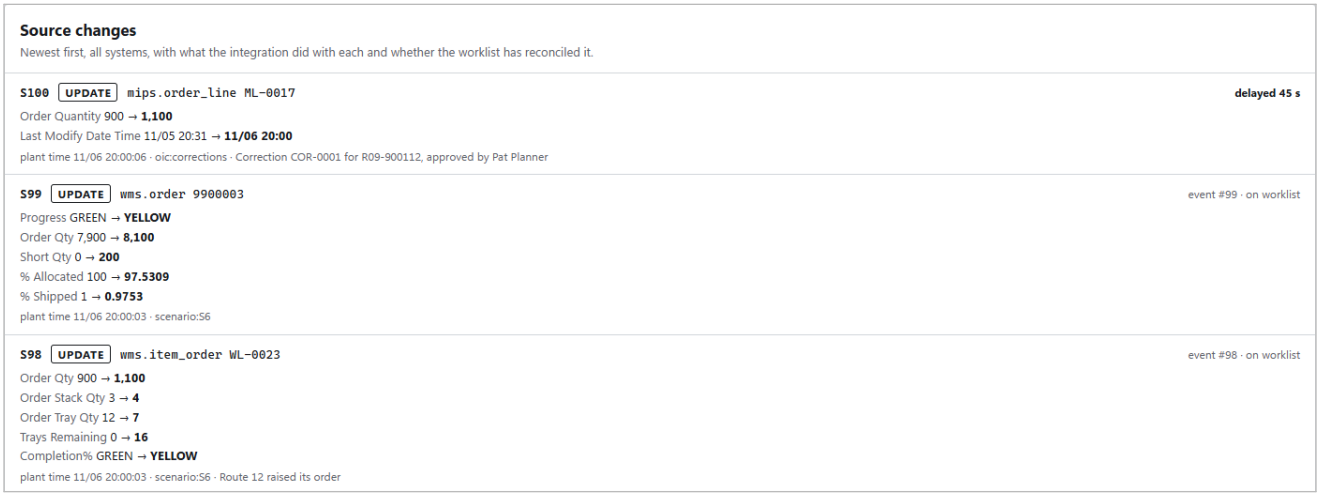


Figure 10. Every source change, newest first, with what changed, who made it, and whether it reached the worklist.

7.3 Making the integration misbehave

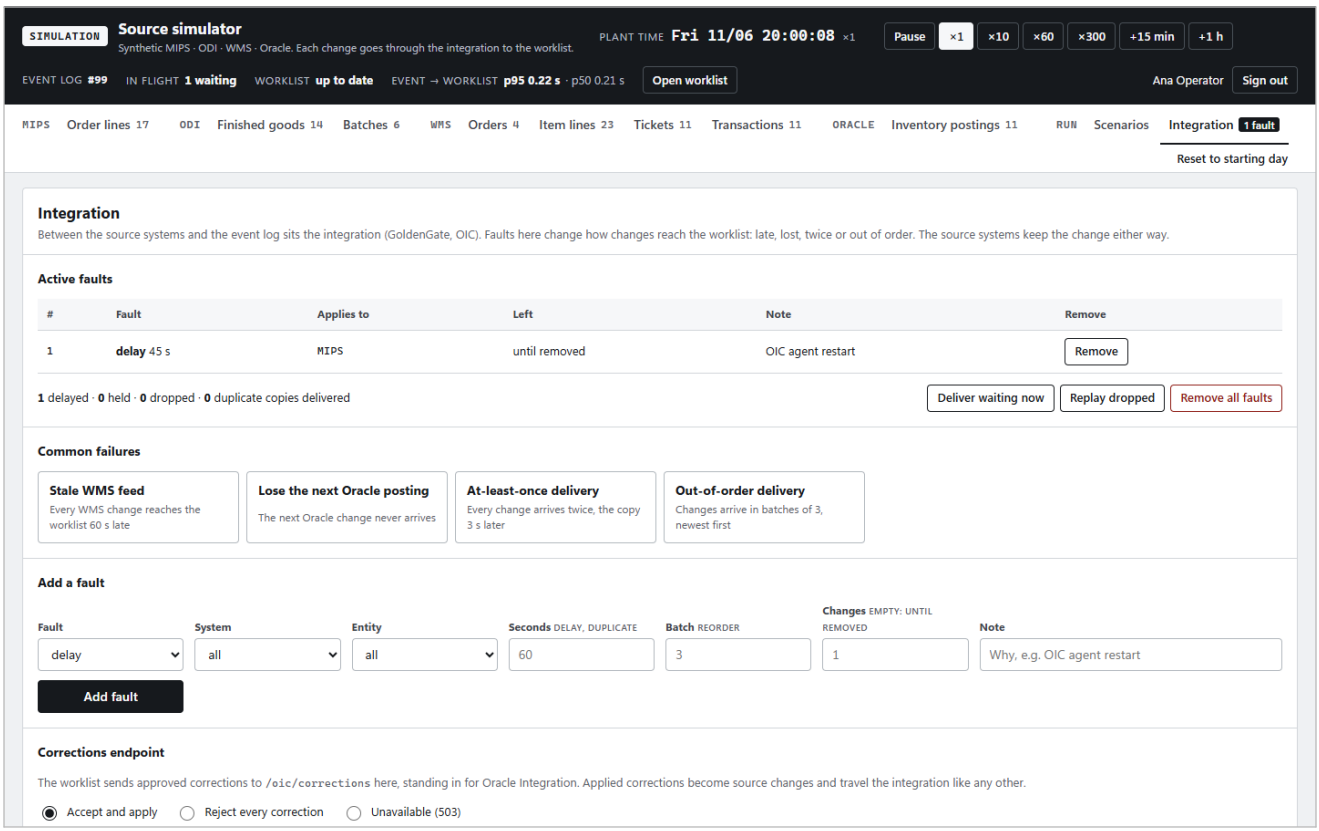


Figure 11. The Integration tab: active faults, the four common failures, a custom fault form, and the corrections endpoint.

Preset	What happens	What the worklist shows
Stale WMS feed	Every WMS change arrives 60 s late	WMS numbers lag; mismatches appear, then clear
Lose the next Oracle posting	One Oracle change never arrives	After 10 plant minutes, R18
At-least-once delivery	Every change arrives twice	Nothing: the copy is recognised and ignored

Preset	What happens	What the worklist shows
Out-of-order delivery	Changes arrive newest first in batches	Nothing: an older value never overwrites a newer one

The corrections endpoint can be set to accept, reject or be unavailable, to show how the worklist handles failed write-backs.

7.4 Scenarios

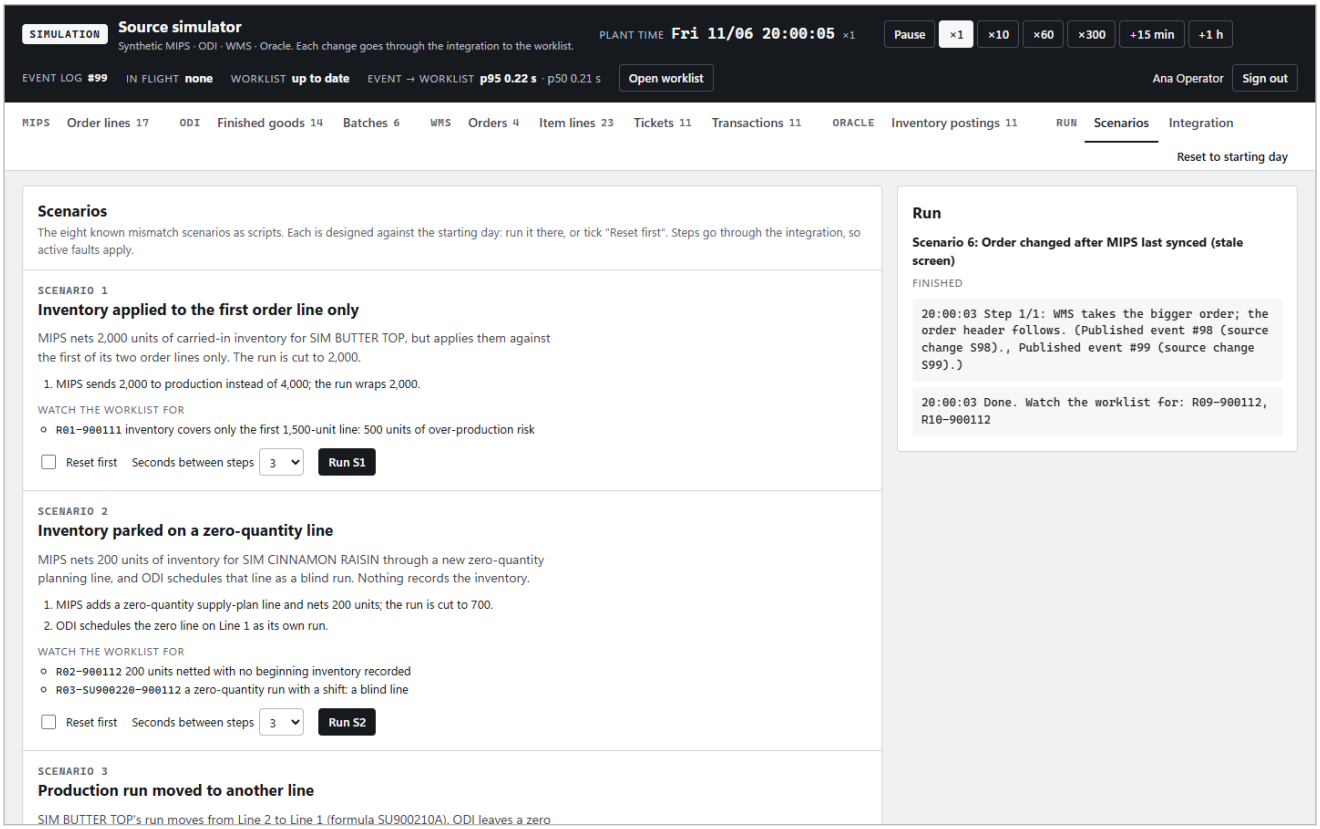


Figure 12. The Scenarios tab: each card describes one known problem, its steps and the flags to watch for. The run log is on the right.

7.5 How one change travels

#	Where	What happens	Time
1	Simulator	The operator raises Route 12's cinnamon-raisin order from 900 to 1,100	instant
2	Integration	Delivered as the next event (a fault could delay, drop, repeat or reorder it)	instant
3	Worklist	Notices the event within half a second and re-runs every rule	under 1 s
4	Worklist	R09 and R10 appear; open screens are told immediately	under 1 s
5	Worklist	About 20 s after things go quiet, the AI writes the explanations	~20 s + model time

8. The practice day and the eight scenarios

The simulation's starting day is one invented Saturday (7 November 2026) with 12 products and 4 customers. Ten products each carry one built-in problem; two are clean, so scenarios can break them live. The agent finds **14 flags** and **$17,500** at risk.

Product	Built-in problem	Flag	$ at risk
900102 Potato	Top-up run made 2,000 when 800 were missing: 1,200 extra	R05 High; R14	$5,628
Plant	Route 12 shipped 7,900 units with no truck scan	R12 High	–
900101 Whole wheat	Stock counted against one line only: 1,000 over-production risk	R01 High	$4,690
900106 Hot-dog buns	Made 3,300, warehouse supplied 4,000: 700 from unseen stock	R07 High	$2,513
900103 White round top	800 units of stock assumed, none recorded	R02	$3,192
900110 Multigrain	Stock 1,500 vs need 1,200: order shows −300	R08	$1,437
900105 Square wheat	Run moved lines, empty copy left behind	R04	–
900108 Honey wheat	Made 2,900 vs need 2,500: 400 not carried forward	R06	$1,796
900107 Hamburger buns	WMS order 3,700 vs plan 3,500; one customer 200 short	R09, R10	$718 each
900109 Italian bread	10 per tray in MIPS, 12 in WMS	R11	–
900104 Round top 2-pack	Empty line inside a live run	R03	–
Plant	7 of 17 plan lines pad the item code with a space	R15	–
900111, 900112	Clean: every system agrees	none	–

The eight scenarios each play one known problem live:

Run	What happens	Flags raised
S1	Butter top: 2,000 leftover units counted against the first line only (plan asks 2,500, need 2,000)	R01 · $1,995
S2	Cinnamon raisin: 200 units of stock netted through a new empty line, then scheduled as a run	R02 · $1,058; R03
S3	Butter top's run moves from Line 2 to Line 1; an empty copy stays	R04
S4	Cinnamon raisin wraps 800 of 900; a top-up run makes 900 more (800 extra)	R05 · $4,232
S5	120 damaged butter-top loaves returned; the receiving ticket still says 4,000	R19 · $478.80; R07 · $478.80
S6	Route 12 raises cinnamon raisin from 900 to 1,100 in WMS; MIPS still plans 900	R09 · $1,058; R10 · $1,058
S7	300 butter-top loaves taken from the freezer; the plan thinks they were baked	R07 · $1,197
S8	A truck ships six products; Oracle rejects one posting and never receives another; after 10 plant minutes both are overdue	R18 · $14,070; R18 · $5,748

Every row, column and calculation behind these flags is in the companion documents SCENARIO_WALKTHROUGH.md (technical) and SCENARIO_WALKTHROUGH_PLAIN.md (plain language), with exact row numbers in the Simulation_Dummy_Data.xlsx workbook.

9. Roles and responsibilities

Role	Responsibilities
Production planner	Reviews flags, accepts or rejects them, approves corrections, corrects MIPS
Shift supervisor	Acknowledges High flags within 15 minutes, claims rows, reads the briefing, leaves notes for the next shift
Plant productivity manager	Receives unacknowledged High flags; in Phase 2, enables automatic write-back for proven rules
IT / integration team	Fixes interface problems: missing or rejected Oracle postings, stale feeds
Master data team	Fixes item data such as units per tray
Business decision-maker	Reads dollars at risk and flag totals to judge the value of the project
Simulator operator	Runs demos and tests: edits data, plays scenarios, injects integration faults

Outputs and reports

Output	For	Contents
Exception worklist	Planners, supervisors, managers	The live list of flags and all actions
Excel workbook (per sales day)	Planners, finance	Briefing, exceptions with a decision column (decisions can be loaded back in), product positions, allocation proposal, rule catalogue, data lineage
HTML dashboard	Managers	A static summary page
JSON file and Reconciliation API	Other systems	Positions, flags, briefing, decisions and corrections in machine-readable form

10. Technology and operations

On the technical side the application is deliberately simple: one Python code base, no separate database in Phase 1, and server-rendered web pages.

Part	Technology
Language and engine	Python 3.12; rules, statistics and learning in plain Python
Web application	FastAPI and Uvicorn; server-rendered pages (Jinja2) with htmx; no JavaScript build
AI	OCI Generative AI, Google Gemini 2.5 Pro, through the OCI Python SDK
Data input	openpyxl for the Excel extracts; an SQLite event log for the simulated feed
State	Decisions and corrections in JSON files; claims and acknowledgments in memory
Packaging	Docker image; Docker Compose for the worklist, the simulator and optional HTTPS (Caddy)
Delivery	GitHub Actions: tests and image build on every change; versioned image release

Security

- Username and password sign-in; passwords stored as PBKDF2-SHA256 hashes (600,000 iterations).
- Signed session cookie, valid 12 hours; a password change or removed account ends the session at once.
- Failed sign-ins are throttled: 5 per username and 50 per network address per 15 minutes.
- Roles checked on the server; only planners and managers can approve write-back.
- Standard browser security headers; requests from other sites are refused.

Running it

Application	Address (this machine)	Used by
Worklist	http://localhost:8001/	Supervisors, planners, managers
Source simulator	http://localhost:8101/	The demo or test operator

Technical detail

Both run as Docker containers and restart with Docker Desktop. The data source is chosen with `RECON_SOURCE` (`excel` or `events`). Server deployment uses Docker Compose with optional automatic HTTPS; see docs/DEPLOYMENT.md. The test suite (49 tests) replays the starting day and all eight scenarios and checks that each raises exactly its expected flags.

11. Open questions for Bimbo IT

The simulated feed makes assumptions about how the real systems will connect. They must be confirmed before Phase 2:

#	Assumption	Why it matters
A1	MIPS schedule lines have a stable row ID that change capture (GoldenGate) can read	Updates and deletes must name the row
A2	The order of a product's MIPS lines is stable	R01 depends on which line is first
A3	ODI runs and batches have stable IDs and are updated in place	As A1, for production
A4	WMS exposes order headers and item lines separately, each with an ID	R09, R10 and R13 compare them
A5	Each system gives a change time and a commit position	Latency, ordering and R18
A6	Column names and meanings match today's extracts	The readers map by column name
A7	Full row images are available (before and after)	A lost change can be recovered
A8	Oracle postings carry the WMS movement number and a status	R18 matches on it
A9	Receipt tickets keep a returned quantity; returns name the ticket	R19 compares them
A10	The 10-minute posting limit is measured from the WMS movement time	Decides when R18 fires
A11	Oracle Integration accepts a row-level correction with a check of the expected old values	Write-back depends on it

A further question: does any system carry a link between a MIPS order line and the production run or warehouse order that fulfils it? If so, reconciliation could move from product totals to individual lines.

12. Limits today and the Phase 2 plan

Limits of Phase 1

- Real data comes from daily Excel extracts, not live connections.
- With the extracts, corrections are recommended, not written back; write-back exists only in the simulation.
- Rules R18 and R19 run only on the simulated feed; the extracts lack the detail they need.
- No SMS or Teams alerts yet.
- Local accounts instead of the company sign-in (OCI IAM).
- Claims and acknowledgments are lost on restart (decisions are kept). One server process only.
- The simulation's invented data follows the client extracts' columns exactly, but some secondary conventions differ (see SOURCE_DATA_MAPPING.md).

Phase 2 plan

Area	Plan
Real-time data	OCI GoldenGate (MIPS), Oracle Integration 3 (WMS, BUSS, Oracle Fusion) and OCI Streaming replace the Excel files. Target: source event to alert in under 2 minutes for 95% of events
Alerts	High: SMS, Teams and worklist, acknowledge in 15 minutes. Medium: Teams and worklist, 2 hours. Low: worklist and shift briefing, end of shift
Write-back	Flagged → Proposed → Approved → Sent → Confirmed, to the real systems
Automatic write-back	Allowed per rule only after 98% planner acceptance over 8 weeks, enabled by the productivity manager
Audit	Each correction has a unique ID and before and after values, so it can be reversed; records kept 2 years
Platform	Containers on OCI Kubernetes Engine behind OCI API Gateway; OCI IAM sign-in; secrets in OCI Vault
Phase 1 exit targets	Planners accept 90% or more of each rule's flags for 4 weeks

13. Glossary

Term	Meaning
Allocated	Set aside in the warehouse for a specific customer order
BUSS / OCloud	The systems that feed customer orders (BUSS) and supply-plan lines (OCloud) into MIPS
Carry forward	Moving surplus units to the next sales day as stock
Confidence	How much the agent trusts a rule, learned from planner decisions (1.0 = not yet judged)
Demand	The total of all MIPS lines for a product
Evidence	The source values that triggered a flag
Flag	One problem found by a rule, with evidence and a recommended correction
Ghost run / blind line	An empty run left behind by a moved run / an empty line inside a live run
Inventory applied	Stock that MIPS subtracted from demand before sending the order to production
MAD	Median absolute deviation: the middle value of the distances from the median
Net requirement	Demand − inventory applied − wrapped + returns: still to make, or surplus if negative
Plant time	The simulation's clock, which the operator can speed up
Reconciled position	One product's figures from all systems, side by side
Recovery run	An extra run to make up a shortfall
Robust z-score	How unusual a value is, measured with the median and MAD so outliers cannot hide
Sales day	The day the product is sold; flags belong to a sales day
SKU	One product, identified by its Oracle finished-good number
Wrapped	Packed at the end of the line: completed production
Write-back	Sending an approved correction from the worklist back to a source system

Appendix A. Rule conditions and calculations

For technical readers: the exact quantities and conditions the engine uses.

Quantities per product

Quantity	How it is calculated
Demand	sum of all the product's MIPS order quantities
Sales-day demand	Demand minus lines moved to the prior sales day
Sales-order demand	Demand on BUSS lines only
Active run	a run with a non-zero ordered or wrapped quantity
Implied inventory	one active run: Demand − net ordered; two or more: Demand − the first run's order; never below 0
Recorded inventory	MIPS beginning inventory + inventory adjustment, over the runs
Consolidated requirement	Demand − implied inventory (not below 0)
First-line risk	(Demand − min(implied inventory, first line)) − consolidated requirement
Net requirement	Demand − implied inventory − wrapped + returns to plant

Rule conditions

Rule	Fires when
R01	2+ MIPS lines and first-line risk above 0
R02	implied inventory above 0, recorded inventory 0, no R01 risk, no negative run
R03	a run with ordered 0 and wrapped 0 that has a shift
R04	a run with ordered 0, wrapped 0 and no shift, whose formula (ignoring a trailing A) produced output on another line
R05	2+ active runs and (wrapped − consolidated requirement) ≥ 100 or (net ordered − demand) ≥ 100
R06	exactly 1 active run, surplus (−net requirement) ≥ 50, no negative run
R07	net requirement ≥ 20 and WMS allocated ≥ sales-day demand
R08	any run with a negative net order
R09	WMS order quantity ≠ sales-day demand
R10	WMS allocated < WMS ordered
R11	MIPS units per container (most common on container 01 lines) ≠ WMS units per container
R12	any WMS order with shipped above 0 and loaded 0
R13	sum of WMS order headers ≠ sum of WMS item lines (extracts also: an item's printed subtotal ≠ its lines)
R14	batches mixed ≠ batches scheduled (scheduled above 0)
R15	any MIPS item code with leading or trailing spaces
R16	sum of the product's ODI MIPSOrderQty ≠ sales-order demand
R17	|robust z| ≥ 3.5 among 8+ values, for wrap overrun per run or final piece variance per batch
R18	a WMS movement with no POSTED Oracle row referencing it, 10+ plant minutes old
R19	returns booked against a ticket exceed the ticket's returned quantity

Related documents

Document	Contents
docs/USER_GUIDE.md	Step-by-step procedures for worklist users
docs/SYSTEMS_GUIDE.md	The worklist and simulator in detail, with guided walkthroughs
docs/TECHNICAL_OVERVIEW.md	Architecture, configuration and code layout
docs/EVENT_CONTRACT.md	The event format, simulated clock, faults and write-back
docs/DEPLOYMENT.md	Server deployment, HTTPS, accounts and releases
docs/simulation_data/	The practice data: workbook, guide, walkthroughs and mapping to client data